

**IMPLEMENTASI SISTEM KEAMANAN INDUSTRI
DENGAN INTEGRASI TIGA ESP MENGGUNAKAN NODE-RED
DAN PEMANTAUAN DATA PAKET MELALUI WIRESHARK**

Dosen Pengampu : **Febrian Hadiatna, S.T., M.T.**
Mata Kuliah : **INTERNET OF THINGS**



Nama Anggota :

Esa Fallah Royani	11-2022-019
Favian Arkan Fadila	11-2023-012
Fahrul Nurfadillah	11-2022-031
Arsyad	
Salman Abiyyu Mufid	11-2022-027
Firza Permana	11-2022-016
Arfinizar Yudhistira	11-2021-036

**PROGRAM STUDI TEKNIK ELEKTRO
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI NASIONAL
BANDUNG**

2025

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Percobaan	2
1.4 Manfaat Percobaan	2
BAB II DASAR TEORI.....	3
2.1 Sistem Keamanan Berbasis IoT	3
2.2 Kontrol Dan Sensor	3
2.3 Protokol MQTT.....	7
2.4 Node-RED sebagai Middleware	8
2.5 Wireshark untuk Analisis Data Paket	8
BAB III METODOLOGI PERCOBAAN	9
3.1 Desain Sistem.....	9
3.2 Perancangan Perangkat Keras.....	11
3.3 Perancangan Perangkat Lunak.....	13
3.4 Pengujian Sistem.....	23
BAB IV HASIL DAN PEMBAHASAN	25
4.1 Dashboard Node-Red	25
4.2 Wireshark	26
BAB V PENUTUP	29
5.1 Kesimpulan	29
DAFTAR PUSTAKA	30
LAMPIRAN	31

DAFTAR GAMBAR

Gambar 1 Komponen RFID.....	3
Gambar 2 Komponen MQ4	4
Gambar 3 Komponen DHT22	4
Gambar 4 ESP 32	5
Gambar 5 ESP8266 (1)	6
Gambar 6 ESP8266 (2)	6
Gambar 7 Logo MQTT	7
Gambar 8 Logo Node Red	8
Gambar 9 Logo Wireshark.....	8
Gambar 10 Diagram Arsitektur	9
Gambar 11 Wiring Esp 8266 (1) dengan RFID	11
Gambar 12 Wiring Rangkaian ESP 8266 dengan MQ4	12
Gambar 13 Wiring ESP 32 Dengan DHT22	12
Gambar 14 Code SSID dan Password	13
Gambar 15 Code Inisialisasi Wifi.....	13
Gambar 16 Setting Alamat Server MQTT dan Client pada ESP8266(1).....	14
Gambar 17 Setting Alamat Server MQTT dan Client pada ESP8266(2).....	14
Gambar 18 Setting Alamat Server MQTT dan Client Pada ESP 32	14
Gambar 19 Code Reconnect untuk MQTT	14
Gambar 20 Code Topic Sensor MQ4	15
Gambar 21 Code Topic Sensor RFID	15
Gambar 22 Code Topic Sensor DHT 22	15
Gambar 23 Code Pembacaan Dan Publish data MQ4.....	16
Gambar 24 Code Perintah Publish MQTT	16
Gambar 25 Code Daftar UID RFID	17
Gambar 26 Code Pembacaan RFID	17
Gambar 27 Code Validasi UID Terbaca	17
Gambar 28 Code Peringatan	18
Gambar 29 Code Pemberhentian Baca UID	18
Gambar 30 Code Publish dan Pembacaan DHT 22	19
Gambar 31 Flow Node RED	19
Gambar 32 Flow Node Red Subscribe MQ4.....	20
Gambar 33 Flow Node Red Subscribe RFID.....	20
Gambar 34 Flow Node Red Subscribe DHT 22 Kelembapan	21
Gambar 35 Function Peringatan ketika Suhu Melebihi 32* C.....	21
Gambar 36 Function Peringatan Akses Masuk.....	22
Gambar 37 Dashboard Monitoring Node RED	22
Gambar 38 Pengujian Dashboard Node RED	23
Gambar 39 Monitoring MQTT ke Node RED	23
Gambar 40 Data Paket Sensor Kelembapan DHT 22	24
Gambar 41 Paket data Sensor MQ4	24
Gambar 42 Paket data Sensor suhu DHT 22	25
Gambar 43 Hasil Tampilan Dasboard Node Red	25

Gambar 44 Hasil Tampilan Peringatan Akses	26
Gambar 45 Lalulintas MQTT ke NodeRed	26
Gambar 46 Hasil Paket Data Kelembapan DHT22	27
Gambar 47 Hasil Paket Data Sensor MQ4	28
Gambar 48 Hasil Paket Data Suhu DHT 22	28

BAB I

PENDAHULUAN

1.1 Latar Belakang

Seiring dengan berkembangnya teknologi, penerapan Internet of Things (IoT) menjadi salah satu elemen penting dalam berbagai sektor, termasuk industri. Dalam sistem keamanan industri, IoT memungkinkan pengawasan dan pengendalian berbagai perangkat secara efisien dan real-time. Salah satu tantangan utama dalam mengimplementasikan sistem berbasis IoT adalah bagaimana mengintegrasikan perangkat yang berbeda untuk berkomunikasi secara efektif dan andal.

Pada percobaan ini, dilakukan implementasi sistem keamanan berbasis IoT dengan menggunakan tiga mikrokontroler ESP32 yang masing-masing dilengkapi sensor: sensor RFID untuk autentikasi akses, sensor gas MQ-4 untuk mendeteksi kebocoran gas, dan sensor DHT22 untuk memantau suhu dan kelembapan. Protokol komunikasi yang digunakan adalah MQTT (Message Queuing Telemetry Transport), yang memungkinkan komunikasi data antar perangkat melalui broker Node-RED. Sistem ini juga dianalisis keamanannya menggunakan Wireshark untuk memastikan stabilitas dan keamanan data.

Percobaan ini bertujuan untuk menghadirkan solusi integrasi IoT yang andal dalam mendukung keamanan industri, dengan memanfaatkan teknologi yang terjangkau, fleksibel, dan dapat dikembangkan lebih lanjut.

1.2 Rumusan Masalah

1. Bagaimana tiga ESP yang dilengkapi sensor berbeda dapat saling berkomunikasi menggunakan protokol MQTT melalui Node-RED?
2. Bagaimana data dari setiap sensor dapat dikumpulkan, diolah, dan dipantau secara real-time melalui Node-RED?

3. Bagaimana cara menganalisis stabilitas komunikasi antar perangkat menggunakan Wireshark?

1.3 Tujuan Percobaan

1. Mengintegrasikan tiga ESP yang dilengkapi sensor berbeda menggunakan protokol MQTT melalui Node-RED.
2. Membuat sistem pemantauan data secara real-time melalui Node-RED dashboard.
3. Menganalisis performa dan keamanan sistem komunikasi antar perangkat menggunakan Wireshark.

1.4 Manfaat Percobaan

1. Memberikan solusi implementasi IoT berbasis multi-sensor untuk meningkatkan keamanan industri.
2. Menyediakan wawasan teknis tentang integrasi protokol MQTT dengan Node-RED dalam mengelola komunikasi antar perangkat IoT.
3. Memberikan panduan teknis dalam menganalisis data jaringan menggunakan Wireshark untuk memastikan keamanan komunikasi data.

BAB II

DASAR TEORI

2.1 Sistem Keamanan Berbasis IoT

Sistem keamanan berbasis IoT mengacu pada integrasi perangkat pintar yang dapat saling berkomunikasi melalui jaringan internet untuk memantau dan mengendalikan berbagai aspek keamanan. Penerapan multi-sensor dalam sistem ini memungkinkan pengawasan yang lebih komprehensif, seperti pengenalan akses, deteksi kebocoran gas, dan pemantauan lingkungan.

Keunggulan sistem keamanan berbasis IoT meliputi efisiensi pemantauan real-time, pengurangan biaya operasional, dan kemampuan untuk merespons ancaman secara cepat. Dalam konteks industri, penerapan ini sangat penting untuk memastikan keselamatan kerja dan melindungi aset perusahaan.

2.2 Kontrol Dan Sensor

2.2.1 Sensor RFID (ESP1)



Gambar 1 Komponen RFID

RFID (Radio Frequency Identification) digunakan untuk autentikasi akses. Teknologi ini memungkinkan identifikasi objek atau individu melalui gelombang radio tanpa kontak fisik. RFID dapat digunakan untuk memverifikasi identitas pengguna sebelum mengakses area tertentu.

2.2.2 Sensor Gas MQ-4 (ESP2)



Gambar 2 Komponen MQ4

Sensor gas MQ-4 dirancang untuk mendeteksi keberadaan gas metana dan gas alam lainnya. Dalam sistem keamanan, sensor ini berfungsi untuk mendeteksi potensi kebocoran gas yang dapat membahayakan lingkungan dan pekerja di area industri.

2.2.3 Sensor DHT22 (ESP3)



Gambar 3 Komponen DHT22

Sensor DHT22 digunakan untuk memantau suhu dan kelembapan. Informasi ini penting dalam pengelolaan lingkungan industri untuk memastikan kondisi yang aman dan nyaman bagi pekerja maupun perangkat elektronik.

2.2.4 ESP 32



Gambar 4 ESP 32

ESP32 adalah mikrokontroler berbasis Wi-Fi dan Bluetooth yang memiliki performa tinggi dan efisiensi daya yang baik. Dalam sistem ini, ESP32 digunakan untuk membaca data dari sensor gas MQ-4. Sensor MQ-4 berfungsi untuk mendeteksi keberadaan gas metana (CH₄) dan gas lain yang serupa di lingkungan. ESP32 memproses data yang diterima dari sensor dan mengirimkannya melalui protokol komunikasi seperti MQTT ke server atau perangkat lain untuk pemantauan dan analisis lebih lanjut.

2.2.5 ESP 8266 (1)



Gambar 5 ESP8266 (1)

ESP8266 adalah modul mikrokontroler dengan konektivitas Wi-Fi yang populer karena ukurannya yang kecil dan kemampuannya dalam mendukung aplikasi berbasis IoT. Pada sistem ini, ESP8266 digunakan untuk membaca data dari modul RFID. Modul RFID digunakan untuk mendeteksi dan mengidentifikasi kartu atau tag RFID yang berisi data unik. Setelah data RFID berhasil dibaca, ESP8266 mengirimkan data tersebut melalui Wi-Fi menggunakan protokol seperti MQTT agar dapat diolah atau divisualisasikan lebih lanjut.

2.2.6 ESP8266 (2)



Gambar 6 ESP8266 (2)

Selain digunakan dengan RFID, ESP8266 juga digunakan untuk membaca data dari sensor DHT22, yang mengukur parameter suhu dan kelembapan. Sensor ini menghasilkan data suhu dalam derajat Celsius dan kelembapan dalam persen. Data tersebut dikirim oleh ESP8266 melalui koneksi Wi-Fi ke server atau perangkat lain, sehingga memungkinkan pemantauan lingkungan secara real-time.

2.3 Protokol MQTT



Gambar 7 Logo MQTT

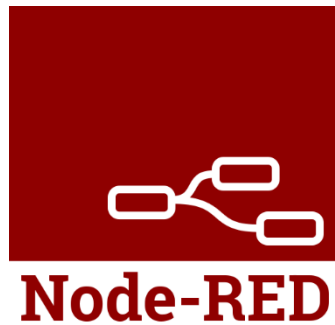
MQTT (Message Queuing Telemetry Transport) adalah protokol komunikasi yang ringan dan efisien, dirancang khusus untuk perangkat dengan sumber daya terbatas. MQTT menggunakan mekanisme publish-subscribe, di mana perangkat penerbit (publisher) mengirim data ke topik tertentu, dan perangkat pelanggan (subscriber) menerima data dari topik tersebut melalui broker.

Keunggulan MQTT meliputi:

1. Penggunaan bandwidth yang rendah.
2. Latensi rendah dalam pengiriman data.
3. Skalabilitas untuk menangani banyak perangkat.

Dalam percobaan ini, MQTT digunakan untuk mengelola komunikasi antar ESP melalui broker Node-RED.

2.4 Node-RED sebagai Middleware



Gambar 8 Logo Node Red

Node-RED adalah platform berbasis alur (flow-based) yang memungkinkan integrasi berbagai perangkat IoT dengan mudah. Dalam sistem ini, Node-RED berfungsi sebagai:

1. Broker MQTT untuk mengelola komunikasi antar ESP.
2. Dashboard visualisasi untuk menampilkan data sensor secara real-time.
3. Pusat pengendalian untuk mengatur alur komunikasi antar perangkat.

2.5 Wireshark untuk Analisis Data Paket



Gambar 9 Logo Wireshark

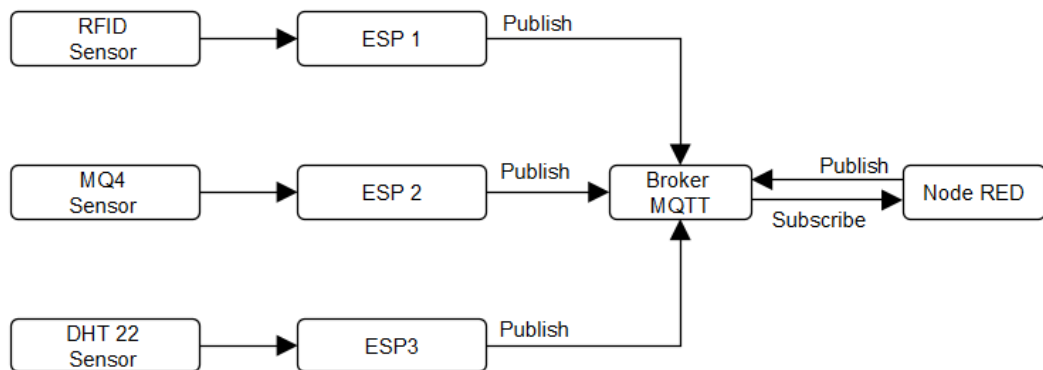
Wireshark adalah alat analisis jaringan yang digunakan untuk memantau dan menganalisis data paket yang dikirimkan melalui jaringan. Dalam konteks penelitian ini, Wireshark digunakan untuk:

1. Menganalisis lalu lintas data antar ESP dan Node-RED.
2. Mengidentifikasi potensi gangguan atau serangan pada protokol MQTT.
3. Mengevaluasi performa komunikasi data antar perangkat

BAB III

METODOLOGI PERCOBAAN

Desain Sistem



Gambar 10 Diagram Arsitektur

Arsitektur Sistem ini dirancang untuk mengintegrasikan tiga perangkat ESP yang masing-masing dilengkapi sensor melalui protokol MQTT dan dikendalikan menggunakan Node-RED. Berikut komponen utama sistem:

1. ESP1: Membaca data RFID

ESP1 berfungsi untuk membaca data dari modul RFID. Modul ini digunakan untuk mendeteksi dan mengidentifikasi kartu atau tag RFID yang mendekat dalam jarak tertentu. Data hasil pembacaan kemudian dikirimkan melalui protokol komunikasi yang telah ditentukan untuk diproses lebih lanjut.

2. ESP2: Membaca data sensor gas MQ-4

ESP2 bertugas membaca data dari sensor gas MQ-4. Sensor ini dirancang untuk mendeteksi keberadaan gas metana (CH₄) dan gas lain yang serupa. Data yang diperoleh dari sensor tersebut akan dikirimkan melalui protokol

komunikasi untuk dianalisis, sehingga dapat digunakan dalam aplikasi monitoring atau sistem peringatan dini.

3. ESP3: Membaca data suhu dan kelembapan menggunakan DHT22

ESP3 berfungsi untuk membaca parameter suhu dan kelembapan menggunakan sensor DHT22. Sensor ini memberikan keluaran data berupa nilai suhu dalam derajat Celsius dan kelembapan dalam persen. Data tersebut kemudian ditransmisikan untuk keperluan monitoring lingkungan atau pengambilan keputusan berbasis IoT.

4. Node-RED: Dashboard visualisasi hasil subscribe dari MQTT

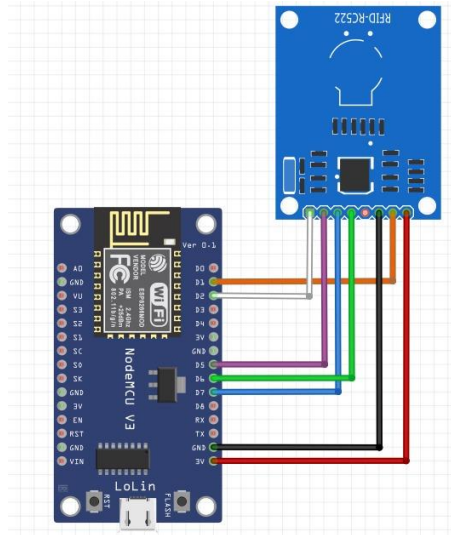
Node-RED digunakan untuk membuat dashboard visual yang menampilkan data hasil subscribe dari topik-topik MQTT. Dashboard ini memungkinkan pengguna untuk melihat data sensor secara real-time, dengan tampilan yang interaktif dan mudah dipahami. Node-RED juga dapat memproses dan mengolah data sebelum ditampilkan pada dashboard.

5. MQTT

- A. Publisher: Perangkat yang mengirimkan data ke suatu topik tertentu. Dalam konteks ini, perangkat seperti ESP1, ESP2, dan ESP3 bertindak sebagai publisher, mengirimkan data yang mereka peroleh ke broker.
- B. Subscriber: Perangkat yang berlangganan ke topik tertentu untuk menerima data. Contohnya, dashboard Node-RED yang berlangganan topik MQTT untuk mendapatkan data dari perangkat.
- C. Broker: Server yang bertindak sebagai perantara antara publisher dan subscriber. Broker mengelola pengiriman pesan dari publisher ke subscriber sesuai topik yang ditentukan, memastikan data sampai dengan andal dan efisien.

Perancangan Perangkat Keras

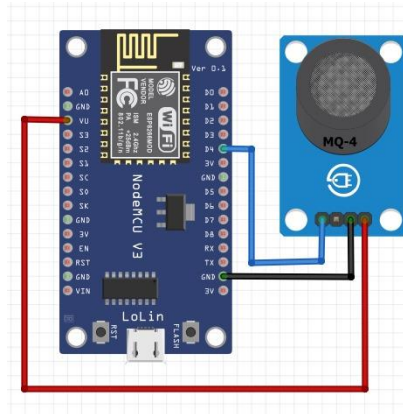
3.2.1 Rangkaian ESP8266 (1) dengan RFID



Gambar 11 Wiring Esp 8266 (1) dengan RFID

Keterangan Pin Rangkaian ESP 8266 (1) dengan EFID	
ESP8266	RFiD
D4	SDA
D5	SCK
D7	MOSI
D6	MISO
-	IRQ
GND	GND
D1	RST
3,3V	3,3V

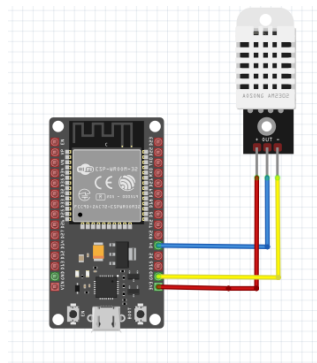
3.2.2 Rangkaian ESP 8266 dengan Sensor Gas MQ-4



Gambar 12 Wiring Rangkaian ESP 8266 dengan MQ4

Keterangan Pin Rangkaian ESP 8266 (2) dengan MQ4	
ESP 8266	MQ4
5 V	VCC
D4	Digital Out
GND	GND

3.2.3 Rangkaian ESP8266 dengan Sensor DHT22



Gambar 13 Wiring ESP 32 Dengan DHT22

Keterangan Pin Rangkaian ESP 32 dengan DHT22	
ESP 8266	MQ4
5 V	VCC
D4	Digital Out

GND	GND
-----	-----

Perancangan Perangkat Lunak

3.3.1 Inisialisasi MQTT

1. Program Dasar untuk Menginisialisasi Komunikasi MQTT di Setiap ESP

Program ini menggunakan library **WiFi** dan **PubSubClient** untuk menghubungkan ESP32 ke jaringan WiFi dan server MQTT. Berikut adalah poin-poin penting terkait inisialisasi komunikasi MQTT:

1. Koneksi Wifi

a. SSID dan Password

```
6  const char* ssid = "Essss";
7  const char* password = "Hex11111";
```

Gambar 14 Code SSID dan Password

Digunakan untuk menyambungkan ke jaringan WiFi

b. Setup Wifi

```
20 void setup_wifi() {
21     delay(10);
22     WiFi.begin(ssid, password);
23     while (WiFi.status() != WL_CONNECTED) {
24         delay(500);
25         Serial.print(".");
26     }
27     Serial.println();
28     Serial.print("WiFi Connected. IP Address: ");
29     Serial.println(WiFi.localIP());
30 }
```

Gambar 15 Code Insialisasi Wifi

Menginisialisasi koneksi WiFi dan menampilkan alamat IP jika berhasil terhubung.

2. Inisialisasi MQTT

a. Alamat Server MQTT dan Client ID

```
8  const char* mqttServer = "test.mosquitto.org";
9  const char* clientID = "favian";
```

Gambar 16 Setting Alamat Server MQTT dan Client pada ESP8266(1)

```
11 const char* mqttServer = "test.mosquitto.org";
12 const char* clientID = "favian";
```

Gambar 17 Setting Alamat Server MQTT dan Client pada ESP8266(2)

```
11 const char* mqttServer = "test.mosquitto.org";
12 const char* clientID = "";
```

Gambar 18 Setting Alamat Server MQTT dan Client Pada ESP 32

mqttServer adalah broker MQTT yang digunakan untuk komunikasi, sedangkan clientID adalah identitas perangkat ESP32

b. Fungsi reconnect()

```
32 void reconnect() {
33     while (!client.connected()) {
34         Serial.print("Attempting MQTT connection...");
35         if (client.connect(clientID)) {
36             Serial.println("connected");
37         } else {
38             Serial.print("failed, rc= ");
39             Serial.print(client.state());
40             Serial.println("try again in 5 seconds");
41             delay(5000);
42         }
43     }
44 }
```

Gambar 19 Code Reconnect untuk MQTT

Bertugas untuk menyambungkan ESP32 ke server MQTT jika koneksi terputus.

3. Topik untuk Publikasi Data Sensor

Topik MQTT tempat data sensor dikirim

```
10  const char* topic = "EAS/gasSensor";
```

Gambar 20 Code Topic Sensor MQ4

```
13  const char* topic = "EAS/RFID";
```

Gambar 21 Code Topic Sensor RFID

```
13  const char* topic1 = "EAS/tempSensor";  
14  const char* topic2 = "EAS/humSensor";
```

Gambar 22 Code Topik Sensor DHT 22

Topik merupakan paket dari setiap sensor yang nanti akan di kirim ke Broker mqtt

2. Code untuk Publikasi Data Sensor

a. MQ4

```

61     client.loop();
62     unsigned long currentMillis = millis();
63     if (currentMillis - previousMillis >= interval) {
64         previousMillis = currentMillis;
65         int gasState = digitalRead(GAS_PIN);
66         if (gasState == LOW) {
67             msg = "Tidak ada gas terdeteksi";
68             Serial.println("Gas Tidak Terdeteksi");
69         } else {
70             msg = "Gas terdeteksi";
71             Serial.println("Gas Terdeteksi");
72         }
73         byte arrSize = msg.length() + 1;
74         char msgC[arrSize];
75         msg.toCharArray(msgC, arrSize);
76         client.publish(topic, msgC);
77     }
78 }

```

Gambar 23 Code Pembacaan Dan Publish data MQ4

Pada pembacaan sensor gas MQ4 di gunakan untu mendeteksi kandungan gas CH₄ (metana) jika terdetekesi maka akan menampilkan status gas terdeteksi dan jika tidak terdeteksi akan menampilkan gas tidak terdeteksi, data dari string tersebut akan di kirmkan ke MQTT beroker melalu code berikut :

```

76     client.publish(topic, msgC);

```

Gambar 24 Code Perintah Publish MQTT

b. RFID

Program ini membaca data UID kartu RFID, memvalidasi UID dengan daftar yang sudah disimpan, dan mengirimkan hasil validasi ke server MQTT Sebgai berikut :

1. ID Tersimpan

```

29  ✓ UIDEntry validUIDs[3] = {
30      { { 0x37, 0x4B, 0x47, 0xD6 }, "Esa" },
31      { { 0x2F, 0x6D, 0xAB, 0xE4 }, "Favian" },
32      { { 0x6F, 0x5E, 0xB7, 0x4D }, "Salman" }
33  };

```

Gambar 25 Code Daftar UID RFID

ID yang tersimpan adalah ID yang di daftarkan dan di simpan di dalam code untuk sebagai pembanding jika ada ID yang baru tapi belum terdaftar

2. Pembacaan UID RFID

```

101      // Periksa apakah ada kartu baru yang terdeteksi
102  ✓   if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
103      // Print UID ke Serial Monitor
104      Serial.print("Card UID: ");
105  ✓   for (byte i = 0; i < rfid.uid.size; i++) {
106      Serial.print(rfid.uid.uidByte[i], HEX);
107      Serial.print(" ");
108      }

```

Gambar 26 Code Pembacaan RFID

Pada Pembacaan Card UID akan di tampiln di serial Monitor berdasarkan code HEX dari Alamat setiap ID kartu

3. Validasi UID

```

112      const char* name = getUIDName(rfid.uid.uidByte);
113
114      if (name != nullptr) { // Jika UID ditemukan
115          accessGranted = true;
116          failedAttempts = 0; // Reset jumlah percobaan gagal
117          Serial.print("Access granted! Welcome, ");
118          Serial.println(name);
119          msg = String(name) + " Memasuki Akses";
120
121          // Publikasikan pesan ke MQTT
122          client.publish(topic, msg.c_str());

```

Gambar 27 Code Validasi UID Terbaca

Untuk validasi data UID di bandingkan dengan data yang muncul di string pada serial monitor jika bernilai true sesuai dengan daftar id yang sudah ada

maka string berupa pesan nama + “memasuki Akses” akan di kirim ke boker mqtt dengan mempilish topic

```
123     } else {  
124         // Kartu salah  
125         accessGranted = false;  
126         failedAttempts++;  
127         Serial.println("Wrong card. Try again.");  
128         msg = "Kartu Salah!, Sisa Percobaan : " + String(3-failedAttempts);  
129         if (failedAttempts >= 3) {  
130             Serial.println("3 failed attempts! Buzzer ON");  
131             msg = "Ada yang mencoba masuk";  
132         }  
133  
134         client.publish(topic, msg.c_str());  
135     }  
136 }  
137 delay(2000);  
138 // Stop reading card until card is removed
```

Gambar 28 Code Peringatan

Jika terdat UID yang tidak terdaftar dan berusha untuk masuk selama 3 kali percobaan maka akan di kirimkan pesan publis ke mqtt broker berupa string “ada yang mencoba masuk”

4. Berhenti membaca UID

```
138 // Stop reading card until card is removed  
139 rfid.PICC_HaltA();  
140 rfid.PCD_StopCrypto1();  
141 msg = "";  
142 client.publish(topic, msg.c_str());  
143 }  
144 }
```

Gambar 29 Code Pemberhentian Baca UID

Pembacaan kartu dilakukan jika kartu menepel pada sensor RFID dan jika tidak maka akan mempublis stirng kosong pada mqtt broker

c. DHT 22

Program ini membaca data Sensor DHT 22 kartu RFID diantaranya Suhu dan Kelembapan dan mengirimkan hasil validasi ke server MQTT Sebgai berikut :

Pembacaan Data Dht 22 dan Publis ke MQTT

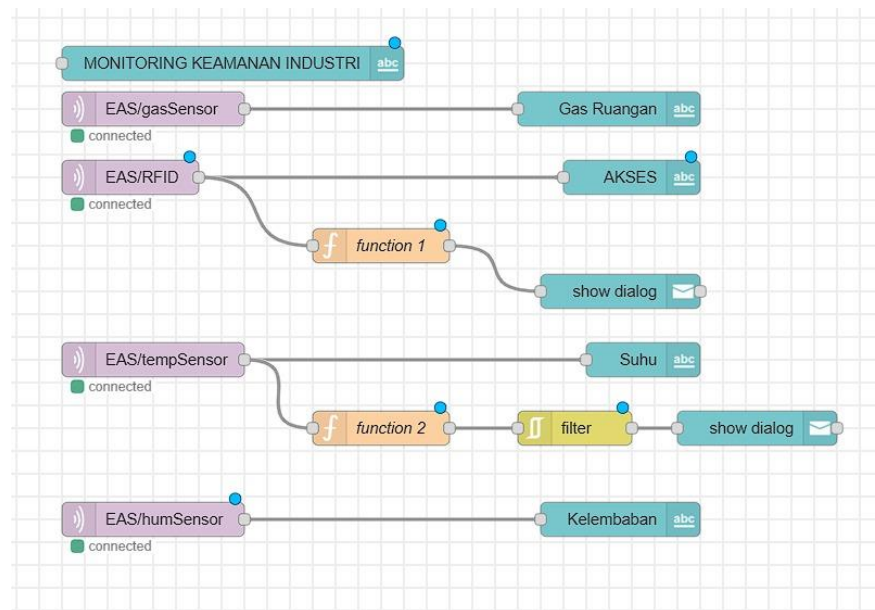
```
if (currentMillis - previousMillis >= interval) {  
  previousMillis = currentMillis;  
  float temperature = dht.readTemperature();  
  float humidity = dht.readHumidity();  
  String payload1 = String(temperature);  
  String payload2 = String(humidity);  
  Serial.print("Temp: "); Serial.print(payload1); Serial.print(" Humidity: "); Serial.println(humidity);  
  client.publish(topic1, payload1.c_str());  
  client.publish(topic2, payload2.c_str());  
}
```

Gambar 30 Code Publish dan Pembacaan DHT 22

Pembacaan DHT 22 dengan parameter temperature dan Humidity di lakukan secara berulang selama 1 detik dan akan terus di publis ke mqtt beroker dengan temperature pada topic1 dan humidity pada topic2

3.3.2 Pengolahan Data di Node-RED

a. flow di Node-RED dari masing-masing ESP.

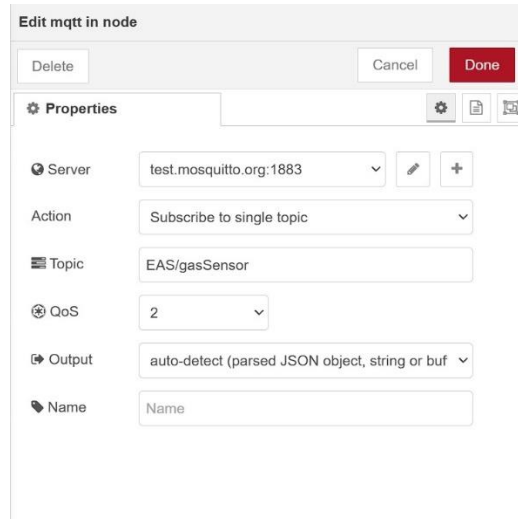


Gambar 31 Flow Node RED

Flow node red secara keseluruhan hampir sama alur dalam menerima data dari MQTT broker lalu menampilkan pada dashboard node red yang di siapkan untuk menampilkan dari masing masing flow, flow

function pada node digunakan untuk logika tambahan yang nantinya akan menampilkan pop up pada dashboard

b. Eas/gasSensor



Edit mqtt in node

Delete Cancel Done

Properties

Server test.mosquitto.org:1883

Action Subscribe to single topic

Topic EAS/gasSensor

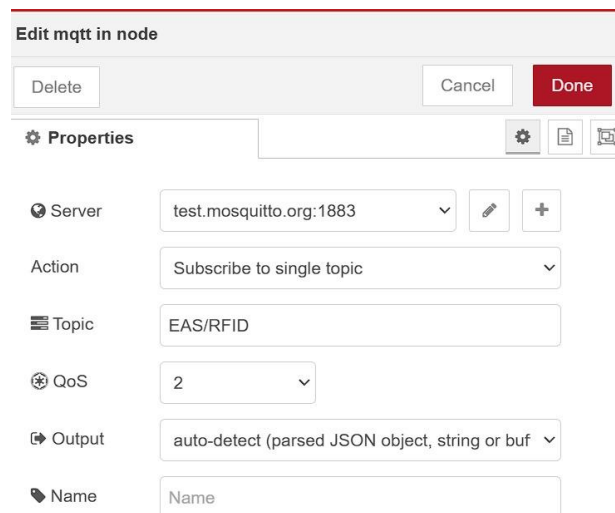
QoS 2

Output auto-detect (parsed JSON object, string or buf)

Name Name

Gambar 32 Flow Node Red Subscribe MQ4

c. EAS/RFID



Edit mqtt in node

Delete Cancel Done

Properties

Server test.mosquitto.org:1883

Action Subscribe to single topic

Topic EAS/RFID

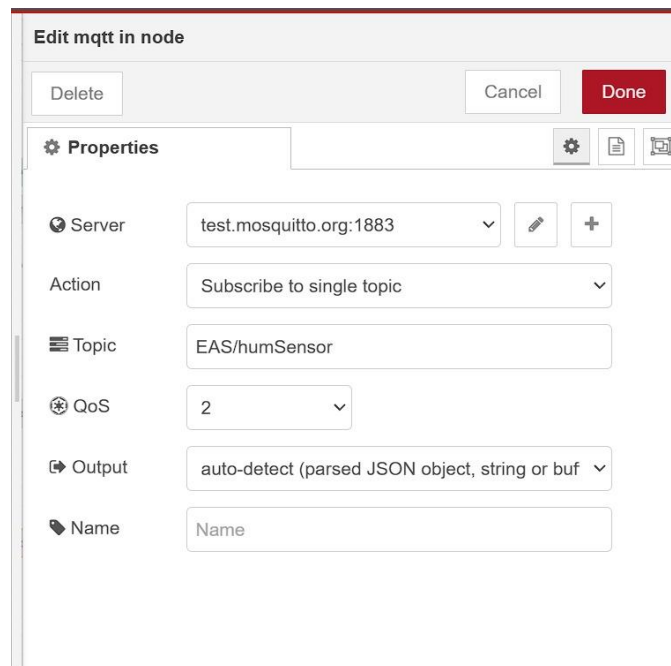
QoS 2

Output auto-detect (parsed JSON object, string or buf)

Name Name

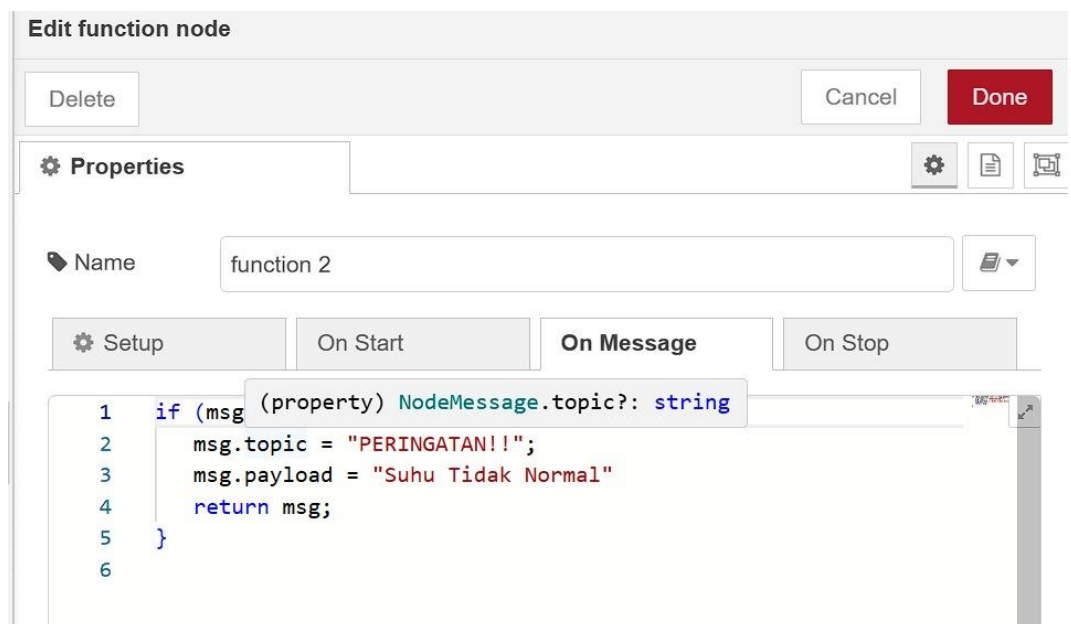
Gambar 33 Flow Node Red Subscribe RFID

d. EAS/humSensor



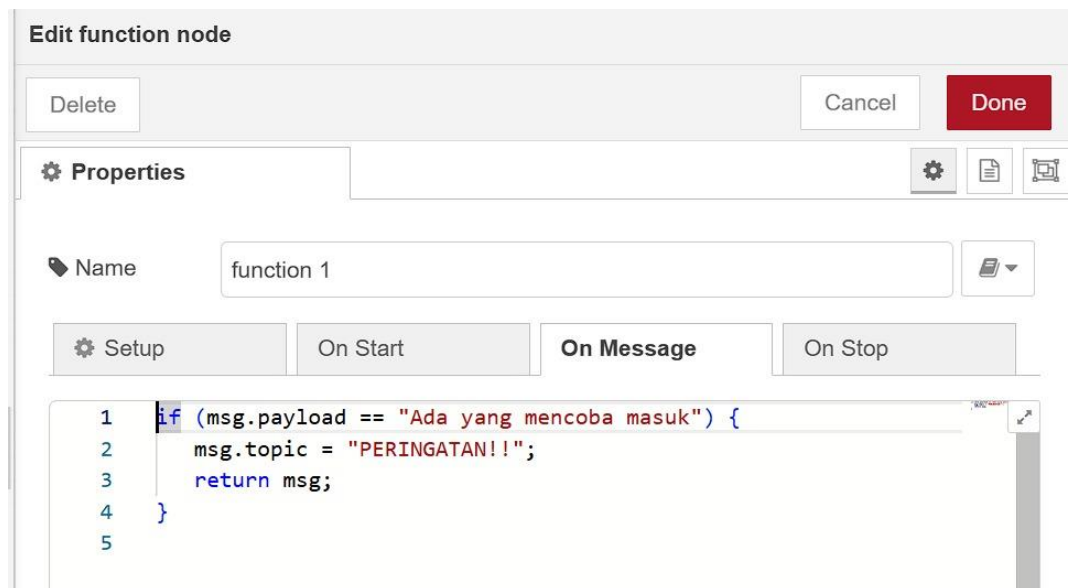
Gambar 34 Flow Node Red Subscribe DHT 22 Kelembapan

e. Function 1



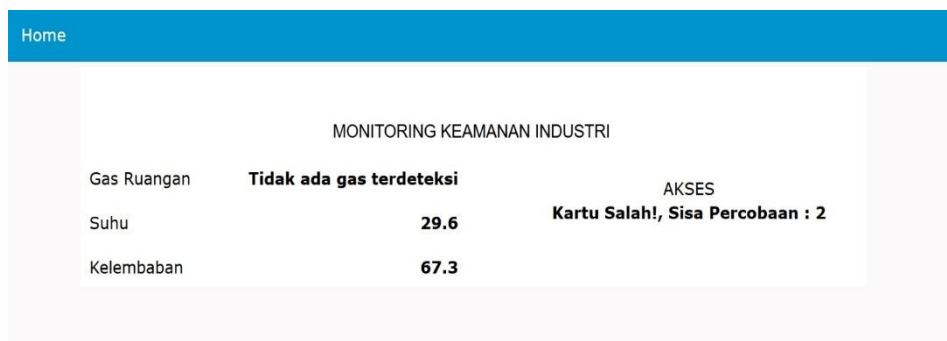
Gambar 35 Function Peringatan ketika Suhu Melebihi 32* C

f. Function 2



Gambar 36 Function Peringatan Akses Masuk

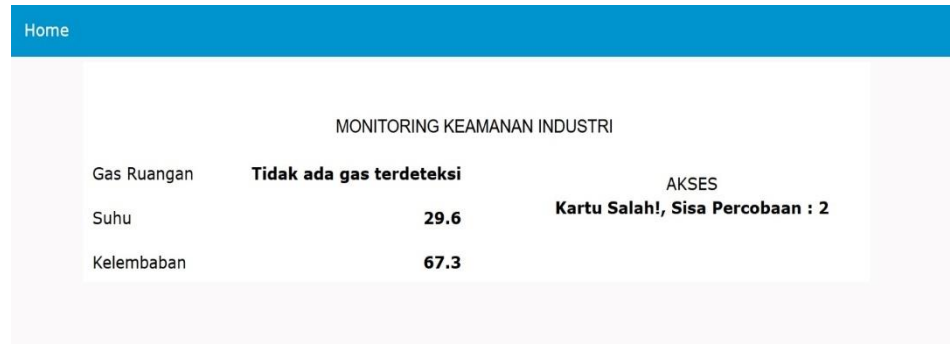
g. dashboard untuk data sensor secara real-time.



Gambar 37 Dashboard Monitoring Node RED

Pengujian Sistem

3.4.1 Dashboard Node-RED



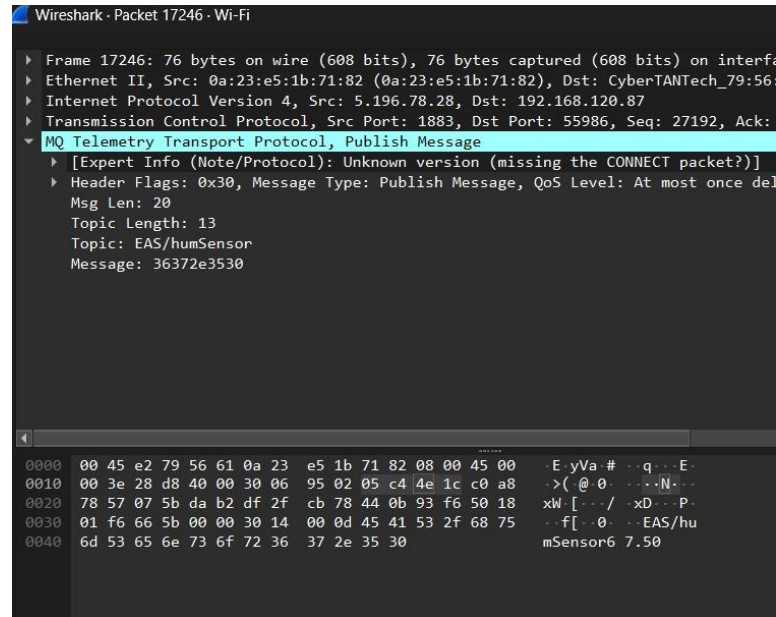
Gambar 38 Pengujian Dashboard Node RED

- Menampilkan data real-time dari RFID, sensor gas MQ-4, dan DHT22.

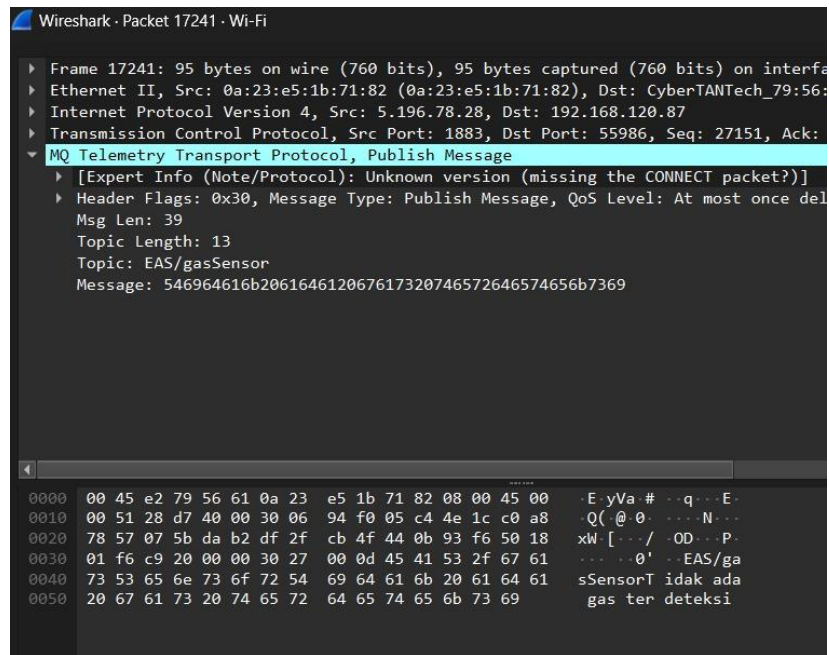
3.4.2 Analisis Data dengan Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
3	0.044285	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
5	0.411208	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
8	1.024191	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
9	1.044569	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
11	1.432369	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
13	2.042174	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
15	2.252772	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
17	2.450631	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
20	3.049988	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
27	3.480009	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
28	3.480009	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
32	4.041144	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
34	4.499746	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
35	4.501084	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
38	5.061298	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
46	5.518926	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
48	5.740852	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
50	6.045793	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
52	6.341739	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]

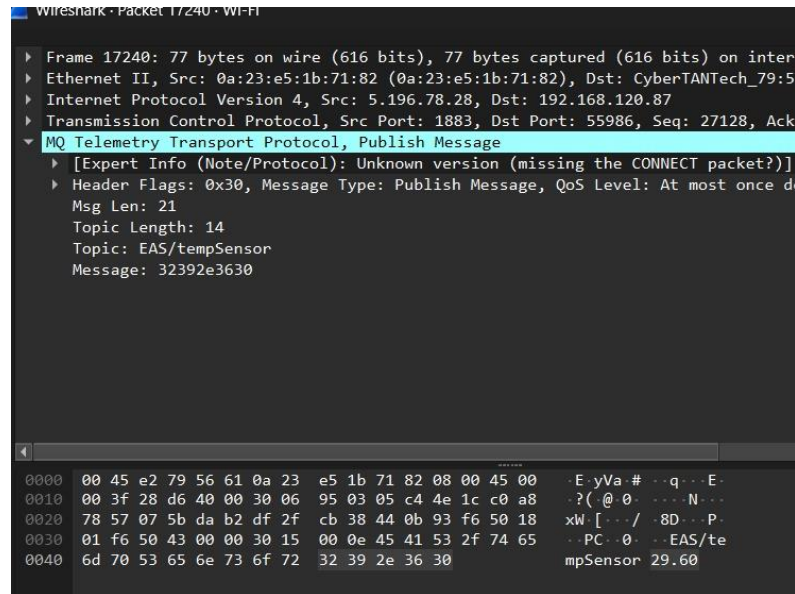
Gambar 39 Monitoring MQTT ke Node RED



Gambar 40 Data Paket Sensor Kelembapan DHT 22



Gambar 41 Paket data Sensor MQ4

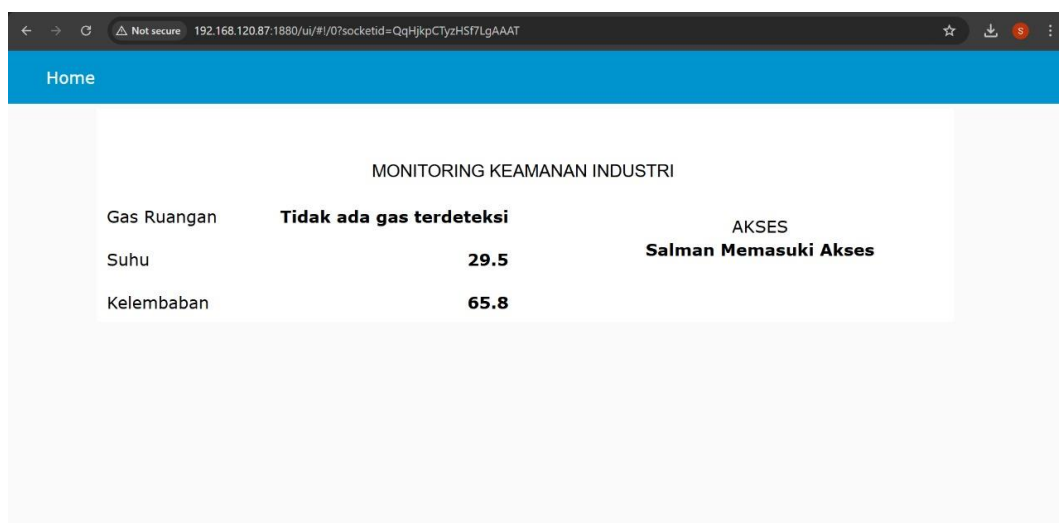


Gambar 42 Paket data Sensor suhu DHT 22

BAB IV

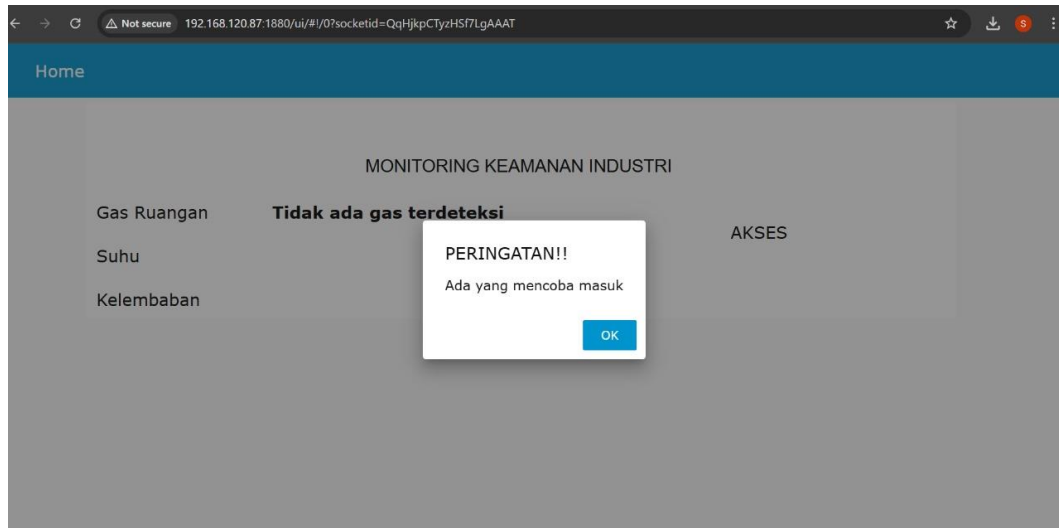
HASIL DAN PEMBAHASAN

4.1 Dashboard Node-Red



Gambar 43 Hasil Tampilan Dasboard Node Red

Pengujian akses kartu menggunakan RFID berjalan dengan baik. Ketika kartu yang telah terdaftar ditempelkan (tap), tampilan pada dashboard Node-RED menampilkan string “Salman Memasuki Akses” pada bagian akses



Gambar 44 Hasil Tampilan Peringatan Akses

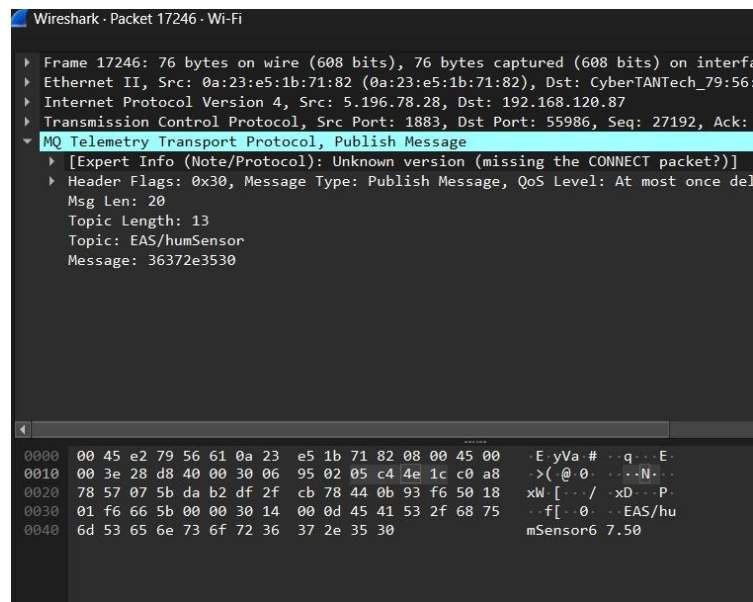
Jika akses ditolak sebanyak tiga kali karena ID kartu tidak terdaftar, akan muncul pop-up peringatan dengan pesan “Ada yang mencoba masuk”. Analisis Komunikasi Data

4.2 Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
3	0.044285	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
5	0.411208	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
8	1.024191	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
9	1.044569	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
11	1.432369	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
13	2.042174	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
15	2.252772	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
17	2.450631	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
20	3.049988	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
27	3.480009	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
28	3.480009	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
32	4.041144	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
34	4.499746	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
35	4.501084	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
38	5.061298	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
46	5.518926	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]
48	5.740852	5.196.78.28	192.168.120.87	MQTT	95	Publish Message [EAS/gasSensor]
50	6.045793	5.196.78.28	192.168.120.87	MQTT	77	Publish Message [EAS/tempSensor]
52	6.341739	5.196.78.28	192.168.120.87	MQTT	76	Publish Message [EAS/humSensor]

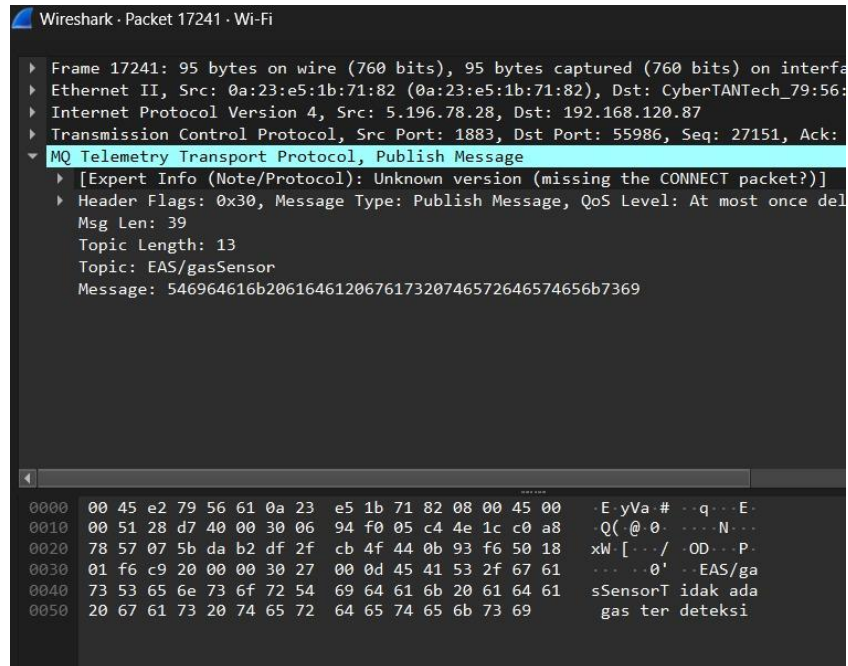
Gambar 45 Lalulintas MQTT ke NodeRed

Lalu lintas protokol MQTT ke Node-RED memiliki alamat masing-masing. Broker MQTT menggunakan alamat 5.196.78.28, dengan tujuan pengiriman data ke Node-RED di alamat 192.168.120.87. Selain itu, dashboard menampilkan informasi *length* dari setiap topik yang terhubung pada masing-masing ESP. Semua akses dari ESP dikumpulkan ke dalam satu broker MQTT dengan alamat yang sama. Jika broker berbeda, sumber data MQTT juga akan berbeda, dan hal ini dapat dilihat melalui *packet capture* menggunakan Wireshark.



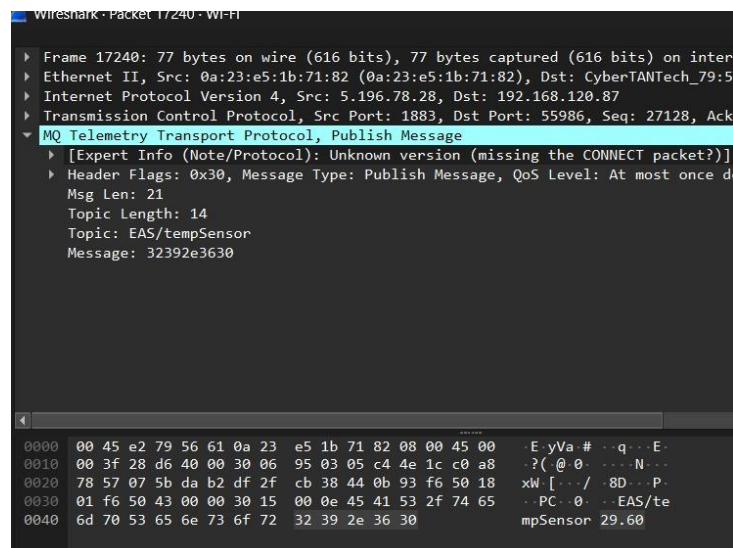
Gambar 46 Hasil Paket Data Kelembapan DHT22

Gambar 46 menunjukkan data paket dari topik kelembapan sensor DHT22 yang dikirim oleh broker MQTT ke tujuan Node-RED.



Gambar 47 Hasil Paket Data Sensor MQ4

Gambar 47 menunjukkan data paket dari topik sensor MQ4 yang dikirim oleh broker MQTT ke tujuan Node-RED.



Gambar 48 Hasil Paket Data Suhu DHT 22

Gambar 48 menunjukkan data paket dari topik suhu sensor DHT22 yang dikirim oleh broker MQTT ke tujuan Node-RED

BAB V

PENUTUP

5.1 Kesimpulan

Hasil pengujian menunjukkan bahwa sistem akses kartu menggunakan RFID berjalan dengan optimal. Sistem dapat mendeteksi kartu yang telah terdaftar dan menampilkan notifikasi *real-time* berupa pesan “*Salman Memasuki Akses*” pada dashboard Node-RED. Selain itu, mekanisme keamanan terbukti efektif dengan adanya pop-up peringatan “*Ada yang mencoba masuk*” ketika percobaan akses dilakukan sebanyak tiga kali menggunakan kartu yang tidak terdaftar.

Lalu lintas data menggunakan protokol MQTT juga terpantau berjalan lancar. Broker MQTT dengan alamat 5.196.78.28 berhasil mengirimkan data dari berbagai perangkat ESP ke Node-RED yang berada pada alamat 192.168.120.87. Semua perangkat ESP terintegrasi dalam satu broker, memastikan pengelolaan data yang terpusat dan efisien. Analisis menggunakan Wireshark membuktikan bahwa data dari topik kelembapan (DHT22), gas (MQ4), dan suhu (DHT22) berhasil dikirim oleh broker MQTT ke Node-RED sesuai tujuan.

Secara keseluruhan, sistem RFID, protokol MQTT, dan dashboard Node-RED telah terintegrasi dengan baik. Hasil ini menunjukkan bahwa sistem mampu berfungsi secara optimal untuk mendukung monitoring dan keamanan dalam jaringan IoT.

DAFTAR PUSTAKA

- Mulyono, S., Qomaruddin, M. and Anwar, M.S. (2018). 'Penggunaan Node-RED pada Sistem Monitoring dan Kontrol Green House berbasis Protokol MQTT', *Jurnal Transistor Elektro dan Informatika (TRANSISTOR EI)*, 3(1), pp. 31–44. Available at: <http://jurnal.unissula.ac.id/online/index.php/EI> (Accessed: 13 January 2025).
- Michael, M., Kallista, M. and Wibawa, I.P.D. (2024). 'Sistem Integrasi dan Implementasi Alat Produksi Biogas Berbasis Internet of Things (IoT)', *e-Proceeding of Engineering*, 11(6), pp. 6748–6754. Available at: [URL jika ada] (Accessed: 13 January 2025).
- Mekongga, I., Mustaziri, M., Fazarrudin, M., Asrafi, I. and Aryanti, A. (2023). 'Prototipe dengan Sistem IoT Pada Pengaturan Suhu dan Kelembaban Pada Permentasi Tempe', *Jurnal Ampere*, 8(2), pp. 127–136. DOI: <http://doi.org/10.31851/ampere>.
- Marpaung, A.N. and Arijanto, R. (2024). 'Rancang Bangun Smart Home IoT dengan Integrasi Kunci RFID dan Otomasi Elektronik', *Bit-Tech (Binary Digital - Technology)*, 7(2), pp. 123–130. DOI: 10.32877/bt.v7i2.1729. Available at: <http://jurnal.kdi.or.id/index.php/bt> (Accessed: 13 January 2025).

LAMPIRAN

TABEL JOB DESKRIPSI EAS IOT

NAMA	TUGAS	DESKRIPSI BAGIAN
Esa Fallah Royani	11-2022-019	Program ESP 1 Sensor RFID, Penyusun Laporan
Favian Arkan Fadila	11-2023-012	Program ESP 2 Sensor DHT 22
Fahrul Nurfadillah Arsyad	11-2022-031	koneksi nodered
Salman Abiyyu Mufid	11-2022-027	dashboard nodered
Firza Permana	11-2022-016	wireshark
Arfinizar Yudhistira	11-2021-036	Program ESP 3 Sensor MQ4

Code ESP 32 dengan dht 22:

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <DHT.h>

#define DHTPIN 4

#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "Essss";

const char* password = "Hex111111";

const char* mqttServer = "test.mosquitto.org"; //broker.mqtt-dashboard.com

const char* clientID = "";

const char* topic1 = "EAS/tempSensor";

const char* topic2 = "EAS/humSensor";

WiFiClient espClient;

PubSubClient client(espClient);
```

```

unsigned long previousMillis = 0;

const long interval = 1000;

String msg = "";

void setup_wifi() {
  delay(10);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("WiFi Connected. IP Address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    if (client.connect(clientID)) {
      Serial.println("connected");
    } else {
      Serial.print("failed, rc= ");
      Serial.print(client.state());
      Serial.println("try again in 5 seconds");
      delay(5000);
    }
  }
}

```

```

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  dht.begin();

  setup_wifi();
  client.setServer(mqttServer, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    String payload1 = String(temperature);
    String payload2 = String(humidity);

    Serial.print("Temp: "); Serial.print(payload1); Serial.print(". Humidity: ");
    Serial.println(humidity);

    client.publish(topic1, payload1.c_str());
    client.publish(topic2, payload2.c_str());
  }
}

```

Code esp8266(1) dengan RFID :

```

#include <SPI.h>

#include <MFRC522.h>

#include <ESP8266WiFi.h>

#include <PubSubClient.h>


#define RST_PIN 5 // RST ke GPIO5 (D1)
#define SS_PIN 4 // SDA ke GPIO4 (D2)


const char* ssid = "Essss";
const char* password = "Hex11111";
const char* mqttServer = "test.mosquitto.org"; // broker.mqtt-dashboard.com
const char* clientID = "favian";
const char* topic = "EAS/RFID";


WiFiClient espClient;
PubSubClient client(espClient);


unsigned long previousMillis = 0;
const long interval = 1000;

String msg = "";


MFRC522 rfid(SS_PIN, RST_PIN);


struct UIDEntry {
    byte uid[4]; // Perbaikan: ubah ukuran dari 3 ke 4
    const char* name;
};


UIDEntry validUIDs[3] = {
    { { 0x37, 0x4B, 0x47, 0xD6 }, "Esa" },

```

```

    { { 0x2F, 0x6D, 0xAB, 0xE4 }, "Favian" },
    { { 0x6F, 0x5E, 0xB7, 0x4D }, "Salman" }
};

```

```

int failedAttempts = 0;

```

```

bool accessGranted = false;

```

```

bool compareUID(byte* uid1, byte* uid2) {
    for (byte i = 0; i < 4; i++) {
        if (uid1[i] != uid2[i]) {
            return false;
        }
    }
    return true;
}

```

```

const char* getUIDName(byte* uid) {
    for (byte i = 0; i < 3; i++) { // Iterasi setiap UID valid
        if (compareUID(uid, validUIDs[i].uid)) {
            return validUIDs[i].name;
        }
    }
    return nullptr; // UID tidak ditemukan
}

```

```

void setup_wifi() {
    delay(10);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }
}

```

```

    Serial.print(".");
}

Serial.println();

Serial.print("WiFi Connected. IP Address: ");
Serial.println(WiFi.localIP());
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        if (client.connect(clientID)) {
            Serial.println("connected");
        } else {
            Serial.print("failed, rc= ");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            delay(5000);
        }
    }
}

void setup() {
    Serial.begin(115200);
    SPI.begin(); // SPI Pins otomatis pada ESP8266
    rfid.PCD_Init();
    Serial.println("RFID RC522 Ready");
    setup_wifi();
    client.setServer(mqttServer, 1883);
}

```

```

void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    unsigned long currentMillis = millis(); // Perbaikan: deklarasi currentMillis

    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;

        // Periksa apakah ada kartu baru yang terdeteksi
        if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
            // Print UID ke Serial Monitor
            Serial.print("Card UID: ");

            for (byte i = 0; i < rfid.uid.size; i++) {
                Serial.print(rfid.uid.uidByte[i], HEX);
                Serial.print(" ");
            }
            Serial.println();

            // Dapatkan nama berdasarkan UID
            const char* name = getUIDName(rfid.uid.uidByte);

            if (name != nullptr) { // Jika UID ditemukan
                accessGranted = true;
                failedAttempts = 0; // Reset jumlah percobaan gagal
                Serial.print("Access granted! Welcome, ");
                Serial.println(name);
                msg = String(name) + " Memasuki Akses";
            }
        }
    }
}

```

```

        // Publikasikan pesan ke MQTT
        client.publish(topic, msg.c_str());
    } else {
        // Kartu salah
        accessGranted = false;
        failedAttempts++;
        Serial.println("Wrong card. Try again.");
        msg = "Kartu Salah!, Sisa Percobaan : " + String(3-failedAttempts);
        if (failedAttempts >= 3) {
            Serial.println("3 failed attempts! Buzzer ON");
            msg = "Ada yang mencoba masuk";
        }

        client.publish(topic, msg.c_str());

    }
    delay(2000);
    // Stop reading card until card is removed
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
    msg = "";
    client.publish(topic, msg.c_str());
}
}
}

```

Code eps8266 dengan MQ4 :

```

#include <WiFi.h>

#include <PubSubClient.h>

```

```

#define GAS_PIN 4

const char* ssid = "Essss";
const char* password = "Hex11111";
const char* mqttServer = "test.mosquitto.org"; //broker.mqtt-dashboard.com
const char* clientId = "favian";
const char* topic = "EAS/gasSensor";

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousMillis = 0;
const long interval = 1000;
String msg = "";
float gas;

void setup_wifi() {
  delay(10);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("WiFi Connected. IP Address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {

```

```

while (!client.connected()) {

  Serial.print("Attempting MQTT connection...");

  if (client.connect(clientID)) {

    Serial.println("connected");

  } else {

    Serial.print("failed, rc= ");

    Serial.print(client.state());

    Serial.println("try again in 5 seconds");

    delay(5000);

  }

}

void setup() {

  // put your setup code here, to run once:

  Serial.begin(9600);

  pinMode(GAS_PIN, INPUT);

  Serial.println("Warming up the MQ4 sensor");

  delay(1000);

  Serial.println("Warming up finished");

  setup_wifi();

  client.setServer(mqttServer, 1883);

}

void loop() {

  if (!client.connected()) {

    reconnect();

  }

  client.loop();

  unsigned long currentMillis = millis();

```

```

if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    int gasState = digitalRead(GAS_PIN);
    if (gasState == LOW) {
        msg = "Tidak ada gas terdeteksi";
        Serial.println("Gas Tidak Terdeteksi");
    } else {
        msg = "Gas terdeteksi";
        Serial.println("Gas Terdeteksi");
    }
    byte arrSize = msg.length() + 1;
    char msgC[arrSize];
    msg.toCharArray(msgC, arrSize);
    client.publish(topic, msgC);
}
}

```

